

Clean Code Principles: Findings & Recommendations

Core idea: Programming can be viewed as a story: variables are characters, functions are actions, classes represent roles, and the program connects those roles to produce a result. Good design keeps that story understandable as the software grows.

The principles below are not rules to follow for their own sake. They are practical techniques for controlling complexity, reducing coupling, and keeping software easy to understand, change, test, and maintain.

1. KISS — Keep It Simple

Use the simplest solution that correctly satisfies the requirement. KISS reduces unnecessary abstraction, cognitive load, debugging effort, and maintenance cost. It does not mean writing the shortest possible code; it means avoiding complexity that the problem does not require.

- Ask: "What is the simplest solution that correctly solves the requirement?"
- Avoid unnecessary frameworks, abstractions, and layers.

2. DRY — Don't Repeat Yourself

Avoid meaningful duplication so that the same rule or logic does not have multiple sources of truth. A shared function or abstraction lets one change update one place. DRY should not be used to force unrelated code into artificial abstractions.

- One change should ideally require one logical place to be updated.
- Remove duplicated knowledge, not merely similar-looking lines.

3. YAGNI — You Aren't Gonna Need It

Do not implement features simply because they might be needed later. Extra features add code, tests, documentation, maintenance, and potential failure points. Build current requirements while keeping the design reasonably extensible.

- Implement what is required now.
- Do not build imaginary future requirements.

4. SOLID — Designing for Change

SOLID principles help object-oriented systems remain understandable and loosely coupled as they grow. They are primarily concerned with responsibilities, extensibility, substitutability, interfaces, and dependencies.

4.1 S — Single Responsibility Principle

A class should have one reason to change. In practical terms, a class should have one focused job. This limits the blast radius of changes and makes classes easier to test and maintain.

- Avoid classes that handle database access, validation, email, reporting, and business logic together.

4.2 O — Open/Closed Principle

Entities should be open for extension and closed for modification. Stable behavior should not need to be rewritten every time a new variation is introduced. This reduces regression risk.

- Prefer extensible designs when there are genuine variations in behavior.

- Do not create elaborate extension mechanisms without a real need.

4.3 L — Liskov Substitution Principle

A subclass must be usable wherever its parent type is expected without causing unexpected behavior. Inheritance should represent a genuine behavioral relationship, not merely code reuse.

- If a caller expects a parent operation to work, a valid subtype should preserve that expectation.

4.4 I — Interface Segregation Principle

Clients should not be forced to depend on methods they do not use. Smaller, focused interfaces reduce dependencies, simplify testing, and make contracts clearer.

- Separate unrelated capabilities instead of creating one oversized interface.

4.5 D — Dependency Inversion Principle

High-level business logic should depend on abstractions rather than concrete infrastructure. Both high- and low-level components can then depend on a stable abstraction. This supports loose coupling, testing, and infrastructure changes.

- Business logic should not unnecessarily know the details of PostgreSQL, an email provider, or another infrastructure implementation.
- Use abstractions where they provide a real boundary.

5. Separation of Concerns

Partition an application into areas with distinct responsibilities. For example, an API can separate HTTP routing, business logic, persistence, and database concerns. This allows one layer to change without unnecessarily rewriting unrelated layers.

- Typical flow: Route → Service → Repository → Database.
- Each layer should have a clear reason to exist.

6. Avoid Premature Optimization

Do not introduce complex performance techniques before there is evidence of a performance problem. Optimization should follow measurement and target a demonstrated bottleneck.

- Prefer: correct → clean → maintainable → measure → identify bottleneck → optimize.
- Optimization that adds complexity without meaningful benefit is a liability.

7. Law of Demeter

Limit how much one unit knows about the internal structure of another. A component should communicate mainly with its immediate collaborators instead of navigating long chains of objects.

- Prefer meaningful operations such as `machine.get_manager_email()` over long chains through internal objects.
- This protects encapsulation and reduces coupling.

How the Principles Connect

These principles reinforce one another rather than operating as isolated rules.

Goal	Principles
Keep code simple	KISS, YAGNI

Goal	Principles
Avoid duplication	DRY
Control responsibilities	SRP, Separation of Concerns
Reduce coupling	SOLID, Law of Demeter
Control technical complexity	Avoid Premature Optimization

Practical Code-Review Questions

- Is this code more complicated than the requirement? → KISS
- Am I repeating the same knowledge or logic? → DRY
- Am I building something without a current requirement? → YAGNI
- Does this class have too many reasons to change? → SRP
- Are unrelated concerns mixed together? → Separation of Concerns
- Are components unnecessarily dependent on each other? → DIP / Law of Demeter
- Can behavior be extended without repeatedly modifying stable code? → OCP
- Does inheritance preserve the expected behavior? → LSP
- Are interfaces forcing unnecessary dependencies? → ISP
- Am I optimizing before measuring a real problem? → Avoid Premature Optimization

Key Takeaway

Good software design is not about writing more sophisticated code; it is about making the required code easy to understand, change, test, and maintain.

The practical outcome is controlled change: when requirements change, the code should require small, localized modifications with minimal impact on unrelated components.